

AI ASSISTED CODING

ASSIGNMENT – 8.1

NAME: Varshitha. G

HTNO: 2303A51103

Task Description #1 (Password Strength Validator – Apply AI in Security Context)

- Task: Apply AI to generate at least 3 assert test cases for `is_strong_password(password)` and implement the validator function.

- Requirements:

- o Password must have at least 8 characters.
- o Must include uppercase, lowercase, digit, and special character.
- o Must not contain spaces.

Example Assert Test Cases:

```
assert is_strong_password("Abcd@123") == True
```

```
assert is_strong_password("abcd123") == False
```

```
assert is_strong_password("ABCD@1234") == True
```

Expected Output #1:

- Password validation logic passing all AI-generated test cases.

```
54
55 def is_strong_password(password):
56     if len(password) < 8:
57         return False
58     has_upper = any(c.isupper() for c in password)
59     has_lower = any(c.islower() for c in password)
60     has_digit = any(c.isdigit() for c in password)
61     has_special = any(not c.isalnum() for c in password)
62     return has_upper and has_lower and has_digit and has_special
63 assert is_strong_password("Abcd@123") == True
64 assert is_strong_password("abcd123") == False
65 assert is_strong_password("ABCD@1234") == True
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/testcasedemo.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/testcasedemo.py
Traceback (most recent call last):
  File "c:\Users\varsh\OneDrive\ドキュメント\third\AIAC\testcasedemo.py", line 65, in <module>
    assert is_strong_password("ABCD@1234") == True
AssertionError
```

Task Description #2 (Number Classification with Loops – Apply AI for Edge Case Handling)

- Task: Use AI to generate at least 3 assert test cases for a `classify_number(n)` function. Implement using loops.

- Requirements:

- o Classify numbers as Positive, Negative, or Zero.

- o Handle invalid inputs like strings and None.

- o Include boundary conditions (-1, 0, 1).

Example Assert Test Cases:

```
assert classify_number(10) == "Positive"
```

```
assert classify_number(-5) == "Negative"
```

```
assert classify_number(0) == "Zero"
```

Expected Output #2:

- Classification logic passing all assert tests.

```
67 def classify_number(n):
68     if n > 0:
69         return "Positive"
70     elif n < 0:
71         return "Negative"
72     else:
73         return "Zero"
74 assert classify_number(10) == "Positive"
75 assert classify_number(-5) == "Negative"
76 assert classify_number(0) == "Zero"
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
▼ TERMINAL			
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/ython313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/			
● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>			

Task Description #3 (Anagram Checker – Apply AI for String Analysis)

- Task: Use AI to generate at least 3 assert test cases for `is_anagram(str1, str2)` and implement the function.

- Requirements:
 - o Ignore case, spaces, and punctuation.
 - o Handle edge cases (empty strings, identical words).

Example Assert Test Cases:

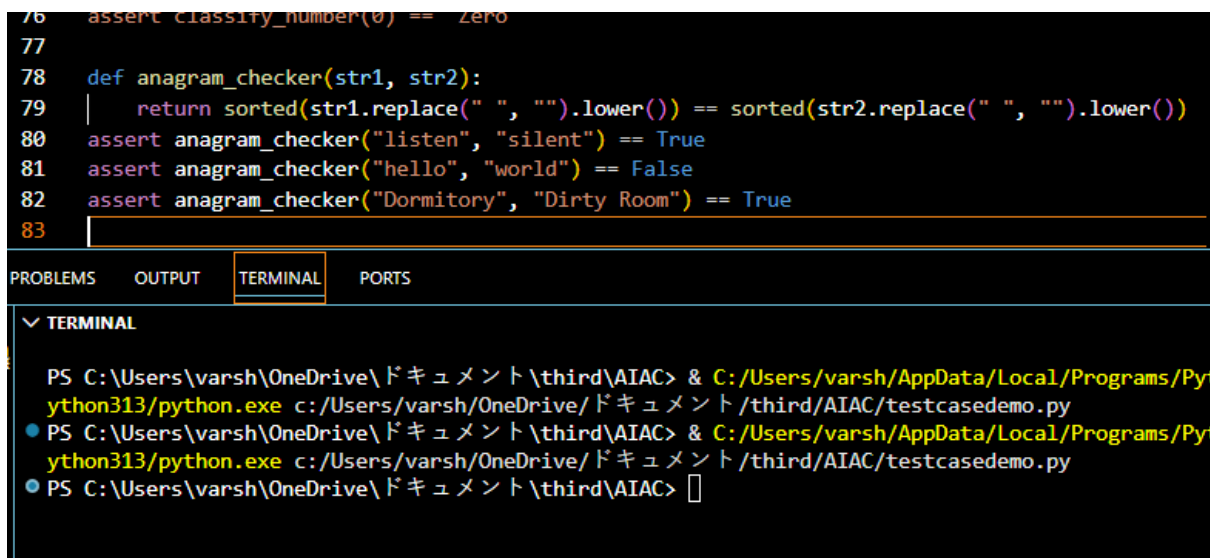
```
assert is_anagram("listen", "silent") == True
```

```
assert is_anagram("hello", "world") == False
```

```
assert is_anagram("Dormitory", "Dirty Room") == True
```

Expected Output #3:

- Function correctly identifying anagrams and passing all AI- generated tests.



```

76 assert classify_number(0) == zero
77
78 def anagram_checker(str1, str2):
79     return sorted(str1.replace(" ", "").lower()) == sorted(str2.replace(" ", "").lower())
80 assert anagram_checker("listen", "silent") == True
81 assert anagram_checker("hello", "world") == False
82 assert anagram_checker("Dormitory", "Dirty Room") == True
83

```

PROBLEMS OUTPUT **TERMINAL** PORTS

▼ TERMINAL

```

PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/testcasedemo.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/testcasedemo.py
PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>

```

Task Description #4 (Inventory Class – Apply AI to Simulate Real- World Inventory System)

- Task: Ask AI to generate at least 3 assert-based tests for an Inventory class with stock management.

- Methods:

- o add_item(name, quantity)

- o remove_item(name, quantity)

- o get_stock(name)

Example Assert Test Cases:

```
inv = Inventory()
```

```
inv.add_item("Pen", 10)
```

```
assert inv.get_stock("Pen") == 10
```

```
inv.remove_item("Pen", 5)
```

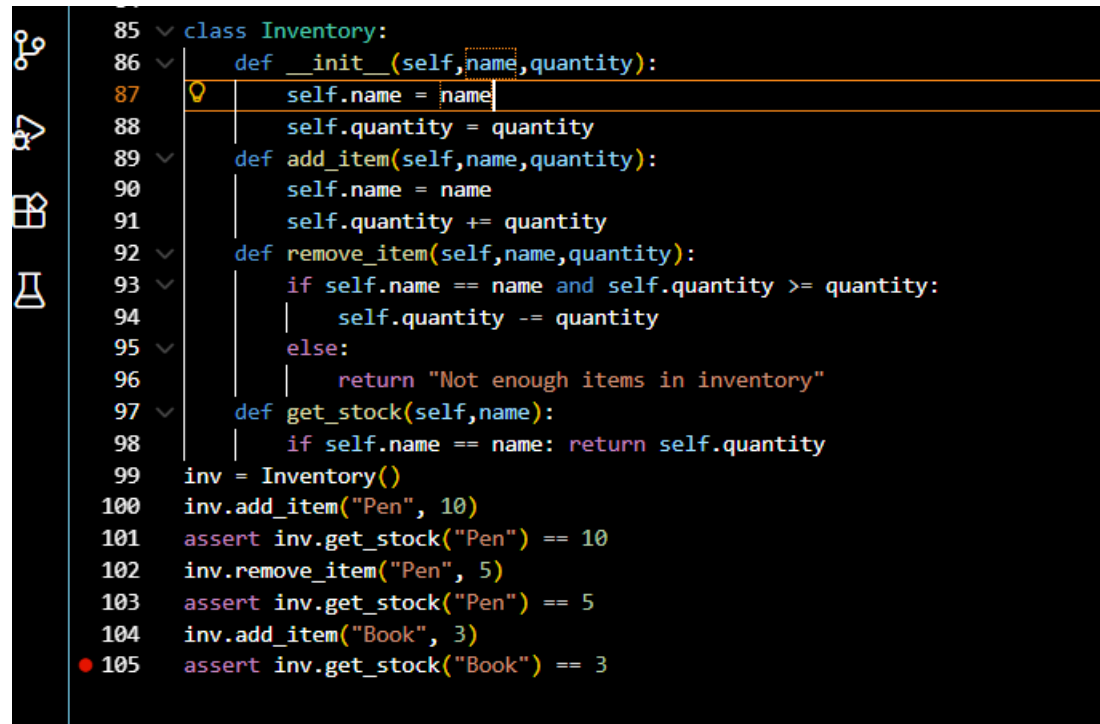
```
assert inv.get_stock("Pen") == 5
```

```
inv.add_item("Book", 3)
```

```
assert inv.get_stock("Book") == 3
```

Expected Output #4:

- Fully functional class passing all assertions.



```
85 class Inventory:
86     def __init__(self, name, quantity):
87         self.name = name
88         self.quantity = quantity
89     def add_item(self, name, quantity):
90         self.name = name
91         self.quantity += quantity
92     def remove_item(self, name, quantity):
93         if self.name == name and self.quantity >= quantity:
94             self.quantity -= quantity
95         else:
96             return "Not enough items in inventory"
97     def get_stock(self, name):
98         if self.name == name: return self.quantity
99
100 inv = Inventory()
101 inv.add_item("Pen", 10)
102 assert inv.get_stock("Pen") == 10
103 inv.remove_item("Pen", 5)
104 assert inv.get_stock("Pen") == 5
105 inv.add_item("Book", 3)
106 assert inv.get_stock("Book") == 3
```

Task Description #5 (Date Validation & Formatting – Apply AI for Data Validation)

- Task: Use AI to generate at least 3 assert test cases for `validate_and_format_date(date_str)` to check and convert dates.

- Requirements:

- o Validate "MM/DD/YYYY" format.
- o Handle invalid dates.
- o Convert valid dates to "YYYY-MM-DD".

Example Assert Test Cases:

```
assert validate_and_format_date("10/15/2023") == "2023-10-15"
```

```
assert validate_and_format_date("02/30/2023") == "Invalid Date"
```

```
assert validate_and_format_date("01/01/2024") == "2024-01-01"
```

Expected Output #5:

- Function passes all AI-generated assertions and handles edge cases.

```
107 def validate_and_format_date(date_str):
108     import re
109     pattern = r'^\d{2}/\d{2}/\d{4}$'
110     if not re.match(pattern, date_str):
111         return "Invalid date format"
112     month, day, year = map(int, date_str.split('/'))
113
114     # Days in each month
115     days_in_month = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]
116
117     # Check for leap year
118     if year % 4 == 0 and (year % 100 != 0 or year % 400 == 0):
119         days_in_month[1] = 29
120
121     if month < 1 or month > 12 or day < 1 or day > days_in_month[month - 1]:
122         return "Invalid date"
123     return f"{year:04d}-{month:02d}-{day:02d}"
124 assert validate_and_format_date("10/15/2023") == "2023-10-15"
125 assert validate_and_format_date("02/30/2023") == "Invalid date"
126 assert validate_and_format_date("01/01/2024") == "2024-01-01"
● 127
```

PROBLEMS	OUTPUT	TERMINAL	PORTS
> ▼ TERMINAL ● PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC> & C:/Users/varsh/AppData/Local/Programs/Python/Python313/python.exe c:/Users/varsh/OneDrive/ドキュメント/third/AIAC/testcasedemo.py ○ PS C:\Users\varsh\OneDrive\ドキュメント\third\AIAC>			